

Comprehensive categorized list of popular transformer pipelines

Task	Pipeline Name	Description
Text Classification	"text-classification"	Classifies text into predefined categories (sentiment, spam, etc.).
Sentiment Analysis	"sentiment-analysis"	Shortcut for text classification on emotions/sentiment.
Token Classification	"token-classification"	Classifies individual tokens (used for NER, POS tagging).
Named Entity Recognition (NER)	"ner"	Shortcut for recognizing entities (person, location, etc.).
Question Answering	"question-answering"	Extracts answer span from a given context paragraph.
Document Question Answering	"document-question-answering"	Performs Q&A on document images or PDFs (multimodal).
Summarization	"summarization"	Generates a concise summary from a long text.
Translation	"translation" or "translation_xx_to_yy"	Translates text between languages.
Text Generation	"text-generation"	Generates text from a prompt using models like GPT.
Conversational AI	"conversational"	For chatbot-style interactions.
Feature Extraction	"feature-extraction"	Returns embeddings for sentences/tokens.
Fill-Mask / Cloze Task	"fill-mask"	Predicts masked words in text.
Zero-Shot Classification	"zero-shot-classification"	Classifies text into unseen labels using natural language inference.

Vision and Multimodal Pipelines

Task	Pipeline Name	Description
Image Classification	"image-classification"	Classifies an image into predefined categories.
Object Detection	"object-detection"	Detects objects and their bounding boxes in images.
Image Segmentation	"image-segmentation"	Produces pixel-level segmentation maps.
Image-to-Text	"image-to-text"	Generates captions or descriptions from images.
Visual Question Answering (VQA)	"vqa"	Answers questions about an image.
Document Question Answering	"document-question-answering"	Combines text and layout (OCR) to answer questions from documents.
Depth Estimation	"depth-estimation"	Predicts depth map of an image.
Image Feature Extraction	"image-feature-extraction"	Generates embeddings from images.

Audio Pipelines

Task	Pipeline Name	Description
Automatic Speech Recognition (ASR)	"automatic-speech-recognition"	Converts speech to text.
Audio Classification	"audio-classification"	Classifies audio samples (music genre, emotion, etc.).
Text-to-Speech (TTS)	"text-to-speech"	Converts text to audio (limited models).
Speaker Diarization	"speaker-diarization"	Identifies and separates different speakers in audio.

Multimodal / Advanced Pipelines

Task	Pipeline Name	Description
CLIP (Text + Image)	"zero-shot-image-classification"	Matches images with text labels using CLIP.
Document Question Answering	"document-question-answering"	Combines OCR + text reasoning for scanned docs.
Visual Question Answering (VQA)	"vqa"	Answers visual questions using image input.
Image-to-Text (Captioning)	"image-to-text"	Generates natural language captions from images.
Video Classification	"video-classification"	Classifies short video clips.

Special and Utility Pipelines

Task	Pipeline Name	Description
Table Question Answering	"table-question-answering"	Answers questions based on tabular data.
Text2Text Generation	"text2text-generation"	Generic text transformation (used for T5, FLAN models).
Zero-Shot Image Classification	"zero-shot-image-classification"	Classifies images using text prompts.
Zero-Shot Audio Classification	"zero-shot-audio-classification"	Classifies audio clips with arbitrary labels.
Sentence Similarity	"sentence-similarity"	Measures semantic similarity between texts.

Example Usage

```
from transformers import pipeline

# Document Question Answering
pipe = pipeline("document-question-answering",
model="impira/layoutlm-document-qa")
result = pipe(question="What is the invoice number?",
image="invoice.png")

# Text Summarization
summarizer = pipeline("summarization", model="facebook/bart-large-
cnn")
summary = summarizer("Long text goes here...", max_length=100,
min_length=30, do_sample=False)

# Zero-Shot Classification
classifier = pipeline("zero-shot-classification",
model="facebook/bart-large-mnli")
classifier("This is an AI course", candidate_labels=["education",
"sports", "politics"])
```

Document Question Answering from Wikipedia (Tkinter + Hugging Face)

How It Works

1. You enter a **Wikipedia URL or topic name** (e.g., [“https://en.wikipedia.org/wiki/Artificial_intelligence”](https://en.wikipedia.org/wiki/Artificial_intelligence) or “Artificial intelligence”).
2. Click **“Fetch Wikipedia Content”** – it loads the first 3000 characters of the page.
3. Enter any **question** (like “*Who coined the term artificial intelligence?*”).
4. Click **“Get Answer”** – it uses Hugging Face’s **DistilBERT QA model** to extract the most relevant answer

pip install transformers wikipedia tkinter

```
import tkinter as tk
from tkinter import scrolledtext, messagebox
from transformers import pipeline
import wikipedia

# Initialize Hugging Face QA pipeline
qa_pipeline = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")

# Function to fetch Wikipedia text
def fetch_wiki_text():
    link = url_entry.get().strip()
    if not link:
        messagebox.showwarning("Input Error", "Please enter a Wikipedia link or topic.")
        return

    try:
        # Extract title from link or topic
        if "wikipedia.org" in link:
            title = link.split("/")[-1]
            title = title.replace("_", " ")
        else:
            title = link

        wiki_page = wikipedia.page(title)
```

```

wiki_text = wiki_page.content[:3000] # limit for performance
context_text.delete(1.0, tk.END)
context_text.insert(tk.END, wiki_text)
messagebox.showinfo("Success", f"Fetched Wikipedia page: {wiki_page.title}")

except Exception as e:
    messagebox.showerror("Error", f"Could not fetch Wikipedia content.\n{e}")

# Function to get the answer from QA model
def get_answer():
    question = question_entry.get().strip()
    context = context_text.get(1.0, tk.END).strip()

    if not question or not context:
        messagebox.showwarning("Missing Input", "Please fetch a Wikipedia article and ask a question.")
        return

    try:
        result = qa_pipeline(question=question, context=context)
        answer = result['answer']
        confidence = result['score'] * 100
        answer_text.delete(1.0, tk.END)
        answer_text.insert(tk.END, f"Answer: {answer}\n\nConfidence: {confidence:.2f}%")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to generate answer.\n{e}")

# Function to close the app
def close_app():
    root.destroy()

# Tkinter GUI setup
root = tk.Tk()
root.title("Wikipedia Document Question Answering")
root.geometry("800x700")
root.config(bg="#f5f5f5")

# Title
tk.Label(root, text="📖 Wikipedia Document Question Answering", font=("Arial", 16, "bold"), bg="#f5f5f5").pack(pady=10)

# Wikipedia Link Input
tk.Label(root, text="Enter Wikipedia Link or Topic:", font=("Arial", 12), bg="#f5f5f5").pack()
url_entry = tk.Entry(root, width=70, font=("Arial", 12))
url_entry.pack(pady=5)

# Fetch Button
tk.Button(root, text="Fetch Wikipedia Content", command=fetch_wiki_text, bg="#4CAF50", fg="white", font=("Arial", 12), width=25).pack(pady=5)

```

```

# Context Area
tk.Label(root, text="Wikipedia Article (Context):", font=("Arial", 12),
bg="#f5f5f5").pack()
context_text = scrolledtext.ScrolledText(root, width=90, height=15, font=("Arial", 10))
context_text.pack(pady=5)

# Question Input
tk.Label(root, text="Enter Your Question:", font=("Arial", 12), bg="#f5f5f5").pack()
question_entry = tk.Entry(root, width=70, font=("Arial", 12))
question_entry.pack(pady=5)

# Submit Button
tk.Button(root, text="Get Answer", command=get_answer, bg="#2196F3", fg="white",
font=("Arial", 12), width=20).pack(pady=10)

# Answer Output
tk.Label(root, text="Answer:", font=("Arial", 12), bg="#f5f5f5").pack()
answer_text = scrolledtext.ScrolledText(root, width=90, height=6, font=("Arial", 11))
answer_text.pack(pady=5)

# Close Button
tk.Button(root, text="Close", command=close_app, bg="#f44336", fg="white", font=("Arial",
12), width=10).pack(pady=10)

root.mainloop()

```

From “Pipeline” to “Agent”

Stage	Description	Example
Static Pipeline	One fixed function (e.g., question-answering).	pipeline("question-answering")
Multi-Step Agent	Breaks a problem into subtasks, reasons which tool to call.	Fetch → Read → Answer

Agentic AI	Adds <i>goal-directed reasoning, memory, and tool-use</i> loops.	"If context insufficient, search more pages."
------------	--	---

So our “agent” will:

1. **Interpret user goal** (question + Wikipedia link/topic)
2. **Decide** which tools to call
3. **Execute** (fetch wiki → QA → return result)
4. Optionally **reflect or re-search** if confidence is low

Hugging Face v4.39+ includes a built-in **transformers.agents** module

```
from transformers import ReactAgent, HfAgent

agent = HfAgent("https://api-inference.huggingface.co/models/bigcode/starcoder")

agent.run("Search Wikipedia for Artificial Intelligence and tell me who coined the term.")
```

```
from transformers import pipeline
import wikipedia

qa_tool = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")

def fetch_wikipedia(topic: str) -> str:
    """Tool 1: Get text from Wikipedia"""
    if "wikipedia.org" in topic:
        title = topic.split("/")[1].replace("_", " ")
    else:
        title = topic
    page = wikipedia.page(title)
    return page.content[:3000]
```

```

def answer_question(context: str, question: str) -> dict:
    """Tool 2: Use Hugging Face QA pipeline"""
    result = qa_tool(question=question, context=context)
    return result

def wiki_agent(task: str, question: str):
    """
    A simple Agent that decides steps:
    1. Fetch context
    2. Run QA
    3. Reflect on confidence
    """
    print(f"[Agent] Step 1: Searching Wikipedia for '{task}' ...")
    context = fetch_wikipedia(task)

    print(f"[Agent] Step 2: Answering question: {question}")
    result = answer_question(context, question)

    print(f"[Agent] Step 3: Confidence = {result['score']:.2f}")
    if result['score'] < 0.3:
        print("[Agent] Low confidence → Could expand search or ask clarifying question.")

    return {
        "answer": result["answer"],
        "confidence": round(result["score"] * 100, 2),
    }
response = wiki_agent(
    task="Artificial intelligence",
    question="Who coined the term artificial intelligence?"
)
print("\n🤖 Agent Response:", response)

```

Agentic AI Dashboard: Wikipedia → Question → Answer

This app:

1. Takes a **Wikipedia topic or link**
2. Takes a **question**
3. Shows agent reasoning steps (**fetch, answer, confidence**)
4. Displays the **final answer**


```

import tkinter as tk
from tkinter import scrolledtext, messagebox
from transformers import pipeline
import wikipedia
import threading
import time

# -----
# Initialize QA Model (Tool)
# -----
qa_tool = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")

# -----
# Agent Tools
# -----
def fetch_wikipedia(topic: str) -> str:
    """Tool 1: Retrieve text from Wikipedia."""
    if "wikipedia.org" in topic:
        title = topic.split("/")[-1].replace("_", " ")
    else:
        title = topic
    page = wikipedia.page(title)
    return page.content[:3000]

def answer_question(context: str, question: str) -> dict:
    """Tool 2: Hugging Face QA."""
    result = qa_tool(question=question, context=context)
    return result

# -----
# Agent Logic
# -----
def wiki_agent(task: str, question: str, log_callback):
    log_callback(f"🔍 Step 1 → Searching Wikipedia for '{task}' ...\n")
    time.sleep(1)
    context = fetch_wikipedia(task)
    log_callback("✅ Fetched Wikipedia content successfully.\n\n")

    log_callback(f"💬 Step 2 → Answering question: {question}\n")
    time.sleep(1)
    result = answer_question(context, question)

    log_callback(f"📊 Step 3 → Confidence = {result['score']:.2f}\n")
    if result["score"] < 0.3:
        log_callback("⚠️ Low confidence - Agent could expand search or ask clarifying question.\n")
    else:
        log_callback("✅ Confidence acceptable.\n")

```

```

    answer = {
        "answer": result["answer"],
        "confidence": round(result["score"] * 100, 2),
    }
    log_callback("\n🤖 Final Answer: " + answer["answer"] +
                f"\n📊 Confidence: {answer['confidence']} %\n")
    return answer

# -----
# Tkinter UI
# -----
root = tk.Tk()
root.title("🤖 Agentic AI Dashboard - Document Question Answering")
root.geometry("850x750")
root.config(bg="#f9f9f9")

title_label = tk.Label(root, text="🤖 Agentic AI Dashboard - Wikipedia Q&A",
                       font=("Segoe UI", 18, "bold"), bg="#f9f9f9")
title_label.pack(pady=10)

tk.Label(root, text="Enter Wikipedia Topic or Link:",
          font=("Segoe UI", 12), bg="#f9f9f9").pack()
topic_entry = tk.Entry(root, width=80, font=("Segoe UI", 12))
topic_entry.pack(pady=5)

tk.Label(root, text="Enter Your Question:",
          font=("Segoe UI", 12), bg="#f9f9f9").pack()
question_entry = tk.Entry(root, width=80, font=("Segoe UI", 12))
question_entry.pack(pady=5)

tk.Label(root, text="Agent Log:",
          font=("Segoe UI", 12), bg="#f9f9f9").pack(pady=(10, 0))
log_box = scrolledtext.ScrolledText(root, width=95, height=25,
                                     font=("Consolas", 10), bg="ffffff", wrap=tk.WORD)
log_box.pack(pady=5)

def log_callback(message):
    log_box.insert(tk.END, message)
    log_box.see(tk.END)
    root.update_idletasks()

def run_agent():
    task = topic_entry.get().strip()
    question = question_entry.get().strip()
    if not task or not question:
        messagebox.showwarning("Missing Input", "Please enter both a Wikipedia topic and question.")
    return

```

```

log_box.delete(1.0, tk.END)
log_callback("🌀 Agent starting ...\n\n")

def threaded_run():
    try:
        wiki_agent(task, question, log_callback)
        log_callback("\n✅ Agent finished.\n")
    except Exception as e:
        log_callback(f"\n❌ Error: {e}\n")

threading.Thread(target=threaded_run, daemon=True).start()

run_button = tk.Button(root, text="Run Agent",
                        command=run_agent, bg="#2196F3", fg="white",
                        font=("Segoe UI", 12, "bold"), width=15)
run_button.pack(pady=10)

tk.Button(root, text="Close", command=root.destroy,
          bg="#f44336", fg="white", font=("Segoe UI", 12),
          width=10).pack(pady=10)

root.mainloop()

```